

Fast Catmull-Rom Spline Interpolation for High-Quality Texture Sampling

Balázs Csébfalvi

Budapest University of Technology and Economics, Hungary

Abstract

It is well known that cubic texture filtering can be efficiently implemented on the GPU by using a method published by Sigg and Hadwiger [SH05], which simplifies the evaluation to a linear combination of linear texture fetches. However, their method cannot be directly applied if the filter kernel takes also negative values like the popular Catmull-Rom spline, for example. In this paper, we propose a modified algorithm that is able to handle also the negative weights. Therefore, using our method, the Catmull-Rom spline interpolation can also be evaluated in one, two, and three dimensions by taking two, four, and eight linear texture fetches, respectively.

CCS Concepts

• **Computing methodologies** → **Image processing; Texturing; Volumetric models;**

1. Introduction

Texture sampling is a fundamental tool in computer graphics as well as in GPU-accelerated image processing and visualization. In computer graphics, it is mostly used for mapping 2D textures to 3D models in virtual environments, while in image processing, images represented by textures are usually transformed by resampling the given texture on a transformed grid. Rescaling images or video streams [RT12] and image registration [SSKH10] are just two representative examples. In GPU-accelerated volume rendering [WE98, KW03], volumetric data is uploaded to the 3D texture memory of the GPU, where it can be efficiently resampled along the viewing rays exploiting the hardwired texture sampling functionality.

In order to define a texture over a continuous domain of texture coordinates, an interpolation between the texels is necessary. Linear interpolation is probably the most frequently used technique because of its efficiency. Nevertheless, using higher-order filters, significantly higher visual quality can be achieved at the cost of a computational overhead. For example, cubic filters such as the cubic B-spline or the Catmull-Rom spline [CR74, Key81, MN88] require to read four, sixteen, and sixty four texels, in 1D, 2D, and 3D, respectively. In contrast, a 1D linear interpolation calculates the weighted sum of two texels, while the bilinear and trilinear interpolations take four and eight texels into account. In order to optimize cubic texture filtering, Sigg and Hadwiger [SH05] proposed a fast GPU implementation exploiting the hardwired linear interpolation such that a cubic filtering is evaluated as a linear combination of linear texture fetches. We will refer to this technique as the SH method throughout this paper. Using the SH method, bicubic

and tricubic B-spline filtering requires only four bilinear and eight trilinear texture fetches, respectively. This results in a significant speed up making cubic texture filtering attractive in practical applications [SKSS08, TUSK09, SK15]. Nevertheless, using an interpolating Catmull-Rom spline kernel instead of the approximating cubic B-spline kernel is problematic, as the SH method assumes that the kernel does not take negative values. This is, however, not the case for the Catmull-Rom spline. The need for an efficient GPU implementation has been indicated by several practitioners in different internet forums, where they also reported that the SH algorithm cannot be trivially adapted to the Catmull-Rom spline interpolation because of the negative weights. To the best of our knowledge, a fast GPU implementation that is able to exploit the hardwired linear interpolation has still not been published or implemented in any framework so far.

2. Related Work

Reconstruction filters have been evaluated in several comparative studies [MN88, MMMY97, ML94, UTY95, LM05, Csé08] in terms of visual quality, numerical accuracy, and efficiency. Because of efficiency reasons, filters are usually designed in 1D, and extended to 2D or 3D by using a separable extension [MMMY97, ML94]. The resulting multidimensional filter inherits the advantageous properties of the 1D kernel. For example, it guarantees the same approximation order [SF71] or becomes interpolating if the 1D kernel is interpolating itself. In computer graphics, the most popular piecewise cubic filters are the cubic B-spline [TBU00] and the Catmull-Rom spline [CR74, Key81]. Both filters are special cases of the BC splines introduced by Mitchell and Netravali [MN88]. The

advantages of these filters are complementary. The Catmull-Rom spline is a C^1 continuous kernel that ensures an approximation order of three. Its greatest advantage is that it interpolates between the original samples and does not require a discrete prefiltering to fulfill the interpolation condition. In contrast, the cubic B-spline filter is not interpolating but approximating. Therefore, it introduces significant blurring. On the other hand, it is C^2 continuous, and using a discrete prefiltering [BTU99], it becomes interpolating perfectly reproducing the original samples. The discrete prefiltering additionally increases its approximation order from two to four [BTU99, Csé08].

3. Motivation

Overall, the cubic B-spline filter is superior over the Catmull-Rom spline filter in terms of visual quality and numerical accuracy but only if it is combined with a relatively expensive deconvolution prefiltering [BTU99] usually performed in the frequency domain [Csé08]. Without prefiltering, the Catmull-Rom spline has been reported to outperform the cubic B-spline [MMY97, Csé08]. There are several applications, where the need for a complex deconvolution prefiltering is clearly a disadvantage because of either the storage or the computational overhead. Therefore, we recommend our method for such application areas as it requires only a simple modulation of the original data rather than a sophisticated prefiltering.

As an example, for a fast and high-quality upscaling of video sequences, Ruijters and Thévenaz [RT12] proposed a prefiltered cubic B-spline interpolation that is entirely implemented on the GPU. Thus, even the prefiltering is performed on the fly on the GPU separately for each frame, which takes around 30 ms for 1024^2 pixels per frame on an nVidia GeForce 9800 GTX [RT12]. However, this is a significant computational overhead compared to the Catmull-Rom spline filtering, which does not require a prefiltering at all to be interpolating. On the other hand, for the Catmull-Rom spline filtering, an efficient implementation that exploits the hardwired linear interpolation like the SH algorithm is not known yet. Therefore, our method clearly fills a gap in the literature as it makes the Catmull-Rom spline filtering as fast on the GPU as the optimized cubic B-spline filtering.

Another example is volume rendering [WE98, KW03], where the segmentation and visualization processes are usually integrated into one interactive system [HCSB]. Therefore it is required not just to efficiently interpolate between the voxels but to rapidly access the original density values in a local neighborhood as well. Using prefiltered cubic B-spline reconstruction, the original densities are not available anymore, so they have to be either additionally stored or reproduced by a relatively expensive filtering step for each voxel. In contrast, using our method, the original densities are reproduced by simply taking the absolute values of the modulated densities. Additionally, as we change only the sign for half of the data values, they can still be stored in the original integer format, which is the case for standard DICOM files, for example. For a prefiltered cubic B-spline reconstruction, however, the prefiltered data values need to be calculated in floating-point format, regardless whether the prefiltering is performed in the spatial domain [BTU99] or in the frequency domain [Csé08].

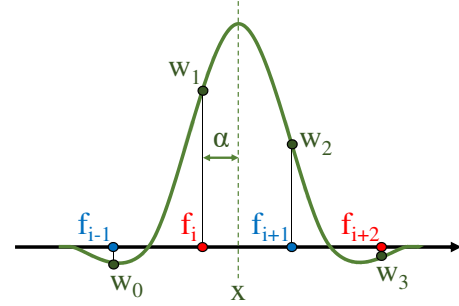


Figure 1: 1D interpolation at position x weighting the four neighboring samples f_{i-1} , f_i , f_{i+1} , f_{i+2} by the Catmull-Rom spline.

4. Assumptions of the SH Method that the Catmull-Rom Spline does not Meet

In 1D, the cubic Catmull-Rom spline interpolation at position x is calculated as the normalized weighted sum of four neighboring samples f_{i-1} , f_i , f_{i+1} , f_{i+2} (see Figure 1):

$$f_{\text{cub}}(x) = f_{i-1}w_0(\alpha) + f_iw_1(\alpha) + f_{i+1}w_2(\alpha) + f_{i+2}w_3(\alpha), \quad (1)$$

where $x = i + \alpha$ for $i \in \mathbb{Z}$ and $\alpha \in [0, 1)$ being the integer and fractional parts of x , respectively. The weights corresponding to the Catmull-Rom spline are defined as follows:

$$\begin{aligned} w_0(\tau) &= -\frac{1}{2}\tau^3 + \tau^2 - \frac{1}{2}\tau, & w_1(\tau) &= \frac{3}{2}\tau^3 - \frac{5}{2}\tau^2 + 1, \\ w_2(\tau) &= -\frac{3}{2}\tau^3 + 2\tau^2 + \frac{1}{2}\tau, & w_3(\tau) &= \frac{1}{2}\tau^3 - \frac{1}{2}\tau^2, \end{aligned} \quad (2)$$

where $\tau \in [0, 1)$. Directly applying the SH method, $f_{\text{cub}}(x)$ is proposed to be evaluated as a linear combination of two terms:

$$\begin{aligned} f_{\text{cub}}(x) &= \\ g_0(\alpha) \cdot \underbrace{\left(f_{i-1} \frac{w_0(\alpha)}{g_0(\alpha)} + f_i \frac{w_1(\alpha)}{g_0(\alpha)} \right)}_{\text{linear interpolation at } i-1 + \frac{w_1(\alpha)}{g_0(\alpha)}} &+ \\ g_1(\alpha) \cdot \underbrace{\left(f_{i+1} \frac{w_2(\alpha)}{g_1(\alpha)} + f_{i+2} \frac{w_3(\alpha)}{g_1(\alpha)} \right)}_{\text{linear interpolation at } i+1 + \frac{w_3(\alpha)}{g_1(\alpha)}} & \\ = g_0(\alpha) \cdot f_{\text{lin}} \left(i-1 + \frac{w_1(\alpha)}{g_0(\alpha)} \right) + g_1(\alpha) \cdot f_{\text{lin}} \left(i+1 + \frac{w_3(\alpha)}{g_1(\alpha)} \right), & \quad (3) \end{aligned}$$

where $g_0(\tau) = w_0(\tau) + w_1(\tau)$, $g_1(\tau) = w_2(\tau) + w_3(\tau)$, and $f_{\text{lin}}(x)$ is a linear interpolation at position x . The trick is to evaluate both terms by separate linear interpolations. The first term is evaluated as a linear interpolation between f_{i-1} and f_i at position $i-1 + \frac{w_1(\alpha)}{g_0(\alpha)}$, while the second term is evaluated as a linear interpolation between f_{i+1} and f_{i+2} at position $i+1 + \frac{w_3(\alpha)}{g_1(\alpha)}$. The first term is then multiplied by $g_0(\alpha)$ and the second term is multiplied by $g_1(\alpha)$ and the two contributions are added. This idea works perfectly if none of the weights are negative, like in case of the cubic B-spline [SH05].

However, for the Catmull-Rom spline, the weights can be negative as $w_0(\alpha) \leq 0$ and $w_3(\alpha) \leq 0$. Therefore, the positions $i - 1 + \frac{w_1(\alpha)}{g_0(\alpha)}$ and $i + 1 + \frac{w_3(\alpha)}{g_1(\alpha)}$ for the first and second linear interpolations can easily fall outside the intervals $[i - 1, i]$ and $[i + 1, i + 2]$, respectively. Consequently, as the Catmull-Rom spline does not meet a fundamental assumption of the SH algorithm, it needs to be re-designed to appropriately handle the negative weights.

5. Our Method

The key idea of this paper is to multiply every second sample by negative one in a preprocessing, and modify the resampling process accordingly to be equivalent to the Catmull-Rom spline interpolation. The modified samples are denoted by c_i and calculated from the original ones as $c_i = f_i \cdot (-1)^i$ (in Figure 1 the blue samples are multiplied by negative one, while the red samples are the original ones if i is assumed to be even). Expressing f_i from c_i as $f_i = c_i \cdot (-1)^i$, and substituting it to Equation 1, we obtain:

$$f_{\text{cub}}(x) = c_{i-1}(-1)^{i-1}w_0(\alpha) + c_i(-1)^iw_1(\alpha) + c_{i+1}(-1)^{i+1}w_2(\alpha) + c_{i+2}(-1)^{i+2}w_3(\alpha). \quad (4)$$

Note that $f_{\text{cub}}(x)$ can be equivalently formulated as

$$f_{\text{cub}}(x) = (-1)^i[-w_0(\alpha)c_{i-1} + w_1(\alpha)c_i - w_2(\alpha)c_{i+1} + w_3(\alpha)c_{i+2}]. \quad (5)$$

Now, similarly to the original SH algorithm, we decompose the evaluation to two separate linear interpolations:

$$\begin{aligned} f_{\text{cub}}(x) = & \left[r_0(\alpha) \cdot \left(c_{i-1} \frac{h_0(\alpha)}{r_0(\alpha)} + c_i \frac{h_1(\alpha)}{r_0(\alpha)} \right) - \right. \\ & \left. r_1(\alpha) \cdot \left(c_{i+1} \frac{h_2(\alpha)}{r_1(\alpha)} + c_{i+2} \frac{h_3(\alpha)}{r_1(\alpha)} \right) \right] (-1)^i = \\ & \left[r_0(\alpha) \cdot c_{\text{lin}} \left(i - 1 + \frac{h_1(\alpha)}{r_0(\alpha)} \right) - r_1(\alpha) \cdot c_{\text{lin}} \left(i + 1 + \frac{h_3(\alpha)}{r_1(\alpha)} \right) \right] \cdot (-1)^i, \end{aligned} \quad (6)$$

where $c_{\text{lin}}(x)$ is a linear interpolation between the modulated samples c_i at position x , and in order to be equivalent to Equation 5, h_0, h_1, h_2, h_3, r_0 , and r_1 are defined as follows:

$$\begin{aligned} h_0(\tau) &= |w_0(\tau)| = -w_0(\tau) = \frac{1}{2}\tau^3 - \tau^2 + \frac{1}{2}\tau, \\ h_1(\tau) &= |w_1(\tau)| = w_1(\tau) = \frac{3}{2}\tau^3 - \frac{5}{2}\tau^2 + 1, \\ h_2(\tau) &= |w_2(\tau)| = w_2(\tau) = -\frac{3}{2}\tau^3 + 2\tau^2 + \frac{1}{2}\tau, \\ h_3(\tau) &= |w_3(\tau)| = -w_3(\tau) = -\frac{1}{2}\tau^3 + \frac{1}{2}\tau^2, \\ r_0(\tau) &= h_0(\tau) + h_1(\tau), \\ r_1(\tau) &= h_2(\tau) + h_3(\tau). \end{aligned} \quad (7)$$

Note that, now the two terms can really be evaluated by two separate linear interpolations as $h_j \geq 0$ for $j \in \{0, 1, 2, 3\}$. Additionally, Equation 6 is completely equivalent to Equation 1 as during

the derivation we have applied only equivalent modifications. Compared to the original SH algorithm formulated by Equation 3, the major differences are the following:

1. We modulate the original samples by $(-1)^i$.
2. We use different weights that are obtained, in fact, by taking the absolute values of the original ones.
3. Instead of addition, we subtract the contribution of the second linear interpolation from that of the first one.
4. The result is multiplied by $(-1)^i$.

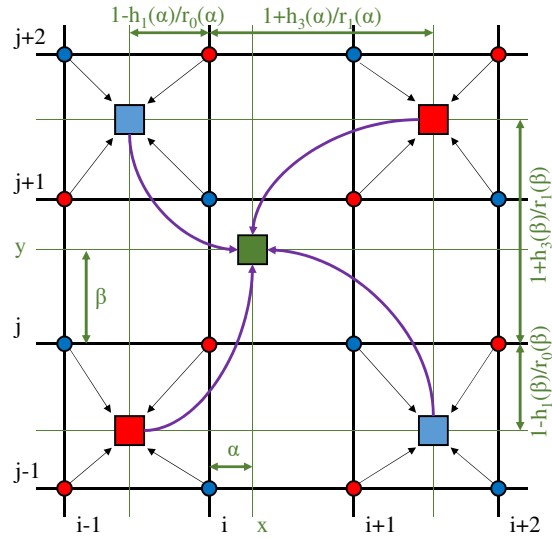


Figure 2: 2D Catmull-Rom spline interpolation at position x, y as a linear combination of four bilinear samples if $i + j$ is assumed to be even. The red circles are the original samples, while the blue samples are the ones that are multiplied by negative one. The red rectangles are bilinear samples, while the blue rectangles are bilinear samples multiplied by negative one. If $i + j$ is assumed to be odd, the red and blue samples swap their roles.

6. 2D Extension

In 2D, the bicubic Catmull-Rom spline interpolation at position x, y is calculated as the normalized weighted sum of sixteen neighboring samples:

$$f_{\text{cub}}(x, y) = \sum_{l=0}^3 \sum_{m=0}^3 f_{i-1+l, j-1+m} w_l(\alpha) w_m(\beta), \quad (8)$$

where $x = i + \alpha$, $y = j + \beta$ for $i, j \in \mathbb{Z}$ and $\alpha, \beta \in [0, 1)$ being the integer and fractional parts of x and y , respectively. Directly applying the SH method, $f(x, y)_{\text{cub}}$ is proposed to be evaluated as a linear

combination of four bilinear interpolations:

$$\begin{aligned}
 f_{\text{cub}}(x, y) = & \left(f_{i-1, j-1} \frac{w_0(\alpha)w_0(\beta)}{g_0(\alpha)g_0(\beta)} + f_{i, j-1} \frac{w_1(\alpha)w_0(\beta)}{g_0(\alpha)g_0(\beta)} \right. \\
 & \left. + f_{i-1, j} \frac{w_0(\alpha)w_1(\beta)}{g_0(\alpha)g_0(\beta)} + f_{i, j} \frac{w_1(\alpha)w_1(\beta)}{g_0(\alpha)g_0(\beta)} \right) \cdot g_0(\alpha)g_0(\beta) + \\
 & \left(f_{i+1, j-1} \frac{w_2(\alpha)w_0(\beta)}{g_1(\alpha)g_0(\beta)} + f_{i+2, j-1} \frac{w_3(\alpha)w_0(\beta)}{g_1(\alpha)g_0(\beta)} \right. \\
 & \left. + f_{i+1, j} \frac{w_2(\alpha)w_1(\beta)}{g_1(\alpha)g_0(\beta)} + f_{i+2, j} \frac{w_3(\alpha)w_1(\beta)}{g_1(\alpha)g_0(\beta)} \right) \cdot g_1(\alpha)g_0(\beta) + \\
 & \left(f_{i-1, j+1} \frac{w_0(\alpha)w_2(\beta)}{g_0(\alpha)g_1(\beta)} + f_{i, j+1} \frac{w_1(\alpha)w_2(\beta)}{g_0(\alpha)g_1(\beta)} \right. \\
 & \left. + f_{i-1, j+2} \frac{w_0(\alpha)w_3(\beta)}{g_0(\alpha)g_1(\beta)} + f_{i, j+2} \frac{w_1(\alpha)w_3(\beta)}{g_0(\alpha)g_1(\beta)} \right) \cdot g_0(\alpha)g_1(\beta) + \\
 & \left(f_{i+1, j+1} \frac{w_2(\alpha)w_2(\beta)}{g_1(\alpha)g_1(\beta)} + f_{i+2, j+1} \frac{w_3(\alpha)w_2(\beta)}{g_1(\alpha)g_1(\beta)} \right. \\
 & \left. + f_{i+1, j+2} \frac{w_2(\alpha)w_3(\beta)}{g_1(\alpha)g_1(\beta)} + f_{i+2, j+2} \frac{w_3(\alpha)w_3(\beta)}{g_1(\alpha)g_1(\beta)} \right) \cdot g_1(\alpha)g_1(\beta) = \\
 & g_0(\alpha)g_0(\beta)f_{\text{lin}}\left(i-1+\frac{w_1(\alpha)}{g_0(\alpha)}, j-1+\frac{w_1(\beta)}{g_0(\beta)}\right) + \\
 & g_1(\alpha)g_0(\beta)f_{\text{lin}}\left(i+1+\frac{w_3(\alpha)}{g_1(\alpha)}, j-1+\frac{w_1(\beta)}{g_0(\beta)}\right) + \\
 & g_0(\alpha)g_1(\beta)f_{\text{lin}}\left(i-1+\frac{w_1(\alpha)}{g_0(\alpha)}, j+1+\frac{w_3(\beta)}{g_1(\beta)}\right) + \\
 & g_1(\alpha)g_1(\beta)f_{\text{lin}}\left(i+1+\frac{w_3(\alpha)}{g_1(\alpha)}, j+1+\frac{w_3(\beta)}{g_1(\beta)}\right),
 \end{aligned} \tag{9}$$

where $f_{\text{lin}}(x, y)$ is a bilinear interpolation at position x, y . Again, the SH algorithm works perfectly for the bicubic B-spline in 2D as well, as it does not introduce negative weights [SH05]. However, it does not work for the bicubic Catmull-Rom spline interpolation as it requires weights $w_1(\alpha)w_0(\beta)$, $w_0(\alpha)w_1(\beta)$, $w_2(\alpha)w_0(\beta)$, $w_3(\alpha)w_1(\beta)$, $w_0(\alpha)w_2(\beta)$, $w_1(\alpha)w_3(\beta)$, $w_3(\alpha)w_2(\beta)$, $w_2(\alpha)w_3(\beta)$ that are either negative or equal to zero. Therefore, the positions of the four bilinear interpolations can easily fall outside the 2D intervals $[i-1, i] \times [j-1, j]$, $[i+1, i+2] \times [j-1, j]$, $[i-1, i] \times [j+1, j+2]$, $[i+1, i+2] \times [j+1, j+2]$, where they are supposed to be evaluated.

In order to appropriately handle the negative weights also in 2D, we propose a modulation of the original samples similarly to the 1D case, but now the samples are multiplied by a 2D discrete function that defines a checkerboard pattern: $c_{i,j} = f_{i,j} \cdot (-1)^{i+j}$. This is illustrated in Figure 2, where the blue circles are the samples multiplied by negative one, while the red samples are the original ones if $i+j$ is assumed to be even. Now we also express $f_{i,j}$ from $c_{i,j}$ as

$f_{i,j} = c_{i,j} \cdot (-1)^{i+j}$ and substitute it to Equation 8:

$$\begin{aligned}
 f_{\text{cub}}(x, y) = & \sum_{l=0}^3 \sum_{m=0}^3 c_{i-1+l, j-1+m} (-1)^{i+j+l+m} w_l(\alpha) w_m(\beta) \\
 = & (-1)^{i+j} \sum_{l=0}^3 \sum_{m=0}^3 c_{i-1+l, j-1+m} (-1)^{l+m} w_l(\alpha) w_m(\beta) = \\
 & (-1)^{i+j} \cdot \left[\left(c_{i-1, j-1} \frac{h_0(\alpha)h_0(\beta)}{r_0(\alpha)r_0(\beta)} + c_{i, j-1} \frac{h_1(\alpha)h_0(\beta)}{r_0(\alpha)r_0(\beta)} \right. \right. \\
 & \left. \left. + c_{i-1, j} \frac{h_0(\alpha)h_1(\beta)}{r_0(\alpha)r_0(\beta)} + c_{i, j} \frac{h_1(\alpha)h_1(\beta)}{r_0(\alpha)r_0(\beta)} \right) \cdot r_0(\alpha)r_0(\beta) - \right. \\
 & \left(c_{i+1, j-1} \frac{h_2(\alpha)h_0(\beta)}{r_1(\alpha)r_0(\beta)} + c_{i+2, j-1} \frac{h_3(\alpha)h_0(\beta)}{r_1(\alpha)r_0(\beta)} \right. \\
 & \left. + c_{i+1, j} \frac{h_2(\alpha)h_1(\beta)}{r_1(\alpha)r_0(\beta)} + c_{i+2, j} \frac{h_3(\alpha)h_1(\beta)}{r_1(\alpha)r_0(\beta)} \right) \cdot r_1(\alpha)r_0(\beta) - \\
 & \left(c_{i-1, j+1} \frac{h_0(\alpha)h_2(\beta)}{r_0(\alpha)r_1(\beta)} + c_{i, j+1} \frac{h_1(\alpha)h_2(\beta)}{r_0(\alpha)r_1(\beta)} \right. \\
 & \left. + c_{i-1, j+2} \frac{h_0(\alpha)h_3(\beta)}{r_0(\alpha)r_1(\beta)} + c_{i, j+2} \frac{h_1(\alpha)h_3(\beta)}{r_0(\alpha)r_1(\beta)} \right) \cdot r_0(\alpha)r_1(\beta) + \\
 & \left(c_{i+1, j+1} \frac{h_2(\alpha)h_2(\beta)}{r_1(\alpha)r_1(\beta)} + c_{i+2, j+1} \frac{h_3(\alpha)h_2(\beta)}{r_1(\alpha)r_1(\beta)} \right. \\
 & \left. + c_{i+1, j+2} \frac{h_2(\alpha)h_3(\beta)}{r_1(\alpha)r_1(\beta)} + c_{i+2, j+2} \frac{h_3(\alpha)h_3(\beta)}{r_1(\alpha)r_1(\beta)} \right) \cdot r_1(\alpha)r_1(\beta) \Big] = \\
 & \left[r_0(\alpha)r_0(\beta)c_{\text{lin}}\left(i-1+\frac{h_1(\alpha)}{r_0(\alpha)}, j-1+\frac{h_1(\beta)}{r_0(\beta)}\right) \right. \\
 & - r_1(\alpha)r_0(\beta)c_{\text{lin}}\left(i+1+\frac{h_3(\alpha)}{r_1(\alpha)}, j-1+\frac{h_1(\beta)}{r_0(\beta)}\right) \\
 & - r_0(\alpha)r_1(\beta)c_{\text{lin}}\left(i-1+\frac{h_1(\alpha)}{r_0(\alpha)}, j+1+\frac{h_3(\beta)}{r_1(\beta)}\right) \\
 & \left. + r_1(\alpha)r_1(\beta)c_{\text{lin}}\left(i+1+\frac{h_3(\alpha)}{r_1(\alpha)}, j+1+\frac{h_3(\beta)}{r_1(\beta)}\right) \right] \cdot (-1)^{i+j},
 \end{aligned}$$

where $c_{\text{lin}}(x, y)$ is a bilinear interpolation between the modulated samples $c_{i,j}$ at position x, y . Note that, similarly to the 1D case, each term can be evaluated by a linear interpolation now as $h_j \geq 0$ for $j \in \{0, 1, 2, 3\}$, but instead of two terms, four terms are evaluated. As a consequence of the checkerboard modulation, two of these four terms are always weighted by negative one. This is illustrated in Figure 2 for the case when $i+j$ is even. The red and blue rectangles denote the four bilinear samples, where we subtract the blue ones from the red ones to obtain the result of the Catmull-Rom spline interpolation at position x, y , which is denoted by a green rectangle. If $i+j$ is assumed to be odd, the red and blue samples simply swap their roles.

7. 3D Extension

In 3D, the tricubic Catmull-Rom spline interpolation at position x, y, z is calculated as the normalized weighted sum of sixty four

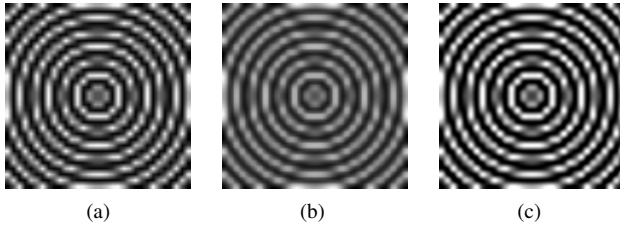


Figure 3: Upsampling of a 2D test signal using bilinear interpolation (a), bicubic B-spline approximation (b), and Catmull-Rom spline interpolation (c).

neighboring samples:

$$f_{\text{cub}}(x, y, z) = \sum_{l=0}^3 \sum_{m=0}^3 \sum_{n=0}^3 f_{i-1+l, j-1+m, k-1+n} w_l(\alpha) w_m(\beta) w_n(\gamma), \quad (11)$$

where $x = i + \alpha$, $y = j + \beta$, $z = k + \gamma$ for $i, j, k \in \mathbb{Z}$ and $\alpha, \beta, \gamma \in [0, 1]$ being the integer and fractional parts of x, y , and z , respectively.

The 3D extension of our interpolation scheme can easily be derived analogously to the 2D case. Therefore, we present only the result of this derivation here:

$$\begin{aligned} f_{\text{cub}}(x, y, z) = & \left[r_0(\alpha) r_0(\beta) r_0(\gamma) c_{\text{lin}} \left(i - 1 + \frac{h_1(\alpha)}{r_0(\alpha)}, j - 1 + \frac{h_1(\beta)}{r_0(\beta)}, k - 1 + \frac{h_1(\gamma)}{r_0(\gamma)} \right) \right. \\ & - r_1(\alpha) r_0(\beta) r_0(\gamma) c_{\text{lin}} \left(i + 1 + \frac{h_3(\alpha)}{r_1(\alpha)}, j - 1 + \frac{h_1(\beta)}{r_0(\beta)}, k - 1 + \frac{h_1(\gamma)}{r_0(\gamma)} \right) \\ & - r_0(\alpha) r_1(\beta) r_0(\gamma) c_{\text{lin}} \left(i - 1 + \frac{h_1(\alpha)}{r_0(\alpha)}, j + 1 + \frac{h_3(\beta)}{r_1(\beta)}, k - 1 + \frac{h_1(\gamma)}{r_0(\gamma)} \right) \\ & + r_1(\alpha) r_1(\beta) r_0(\gamma) c_{\text{lin}} \left(i + 1 + \frac{h_3(\alpha)}{r_1(\alpha)}, j + 1 + \frac{h_3(\beta)}{r_1(\beta)}, k - 1 + \frac{h_1(\gamma)}{r_0(\gamma)} \right) \\ & - r_0(\alpha) r_0(\beta) r_1(\gamma) c_{\text{lin}} \left(i - 1 + \frac{h_1(\alpha)}{r_0(\alpha)}, j - 1 + \frac{h_1(\beta)}{r_0(\beta)}, k + 1 + \frac{h_3(\gamma)}{r_1(\gamma)} \right) \\ & + r_1(\alpha) r_0(\beta) r_1(\gamma) c_{\text{lin}} \left(i + 1 + \frac{h_3(\alpha)}{r_1(\alpha)}, j - 1 + \frac{h_1(\beta)}{r_0(\beta)}, k + 1 + \frac{h_3(\gamma)}{r_1(\gamma)} \right) \\ & + r_0(\alpha) r_1(\beta) r_1(\gamma) c_{\text{lin}} \left(i - 1 + \frac{h_1(\alpha)}{r_0(\alpha)}, j + 1 + \frac{h_3(\beta)}{r_1(\beta)}, k + 1 + \frac{h_3(\gamma)}{r_1(\gamma)} \right) \\ & \left. - r_1(\alpha) r_1(\beta) r_1(\gamma) c_{\text{lin}} \left(i + 1 + \frac{h_3(\alpha)}{r_1(\alpha)}, j + 1 + \frac{h_3(\beta)}{r_1(\beta)}, k + 1 + \frac{h_3(\gamma)}{r_1(\gamma)} \right) \right] \\ & \cdot (-1)^{i+j+k}, \end{aligned} \quad (12)$$

where $c_{\text{lin}}(x, y, z)$ is a trilinear interpolation between the modulated samples $c_{i,j,k}$ at position x, y, z . Note that, now the samples are multiplied by a 3D discrete function that defines a 3D checkerboard pattern: $c_{i,j,k} = f_{i,j,k} \cdot (-1)^{i+j+k}$.

8. GPU Implementation

Our method for fast tricubic Catmull-Rom spline interpolation can be directly implemented by the following GLSL code:

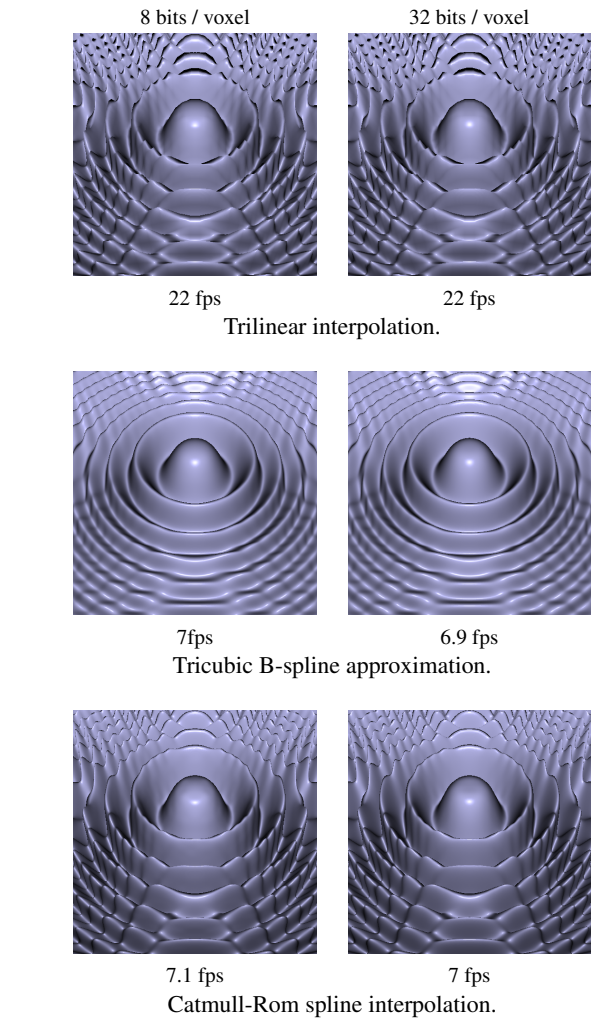


Figure 4: Reconstruction of the Marschner-Lobb test signal using trilinear interpolation, tricubic B-spline approximation, and Catmull-Rom spline interpolation. Note that, compared to the Catmull-Rom spline interpolation, the trilinear interpolation results in much more apparent postaliasing artifacts, while the tricubic B-spline approximation introduces a stronger blurring effect.

```
float CatmullRomSplineInterpolation(
    vec3 samplePosition) {
    vec3 s = samplePosition * size - 0.5;
    vec3 i = floor(s);
    vec3 t = s - i;

    vec3 t2 = t * t;
    vec3 t3 = t2 * t;

    vec3 h0 = 0.5 * t3 - t2 + 0.5 * t;
    vec3 h1 = 1.5 * t3 - 2.5 * t2 + 1.0;
    vec3 h2 = -1.5 * t3 + 2.0 * t2 + 0.5 * t;
    vec3 h3 = -0.5 * t3 + 0.5 * t2;

    vec3 r0 = h0 + h1;
```

```

vec3 r1 = h2 + h3;

vec3 p0 = (h1 / r0 + i - 0.5) / size;
vec3 p1 = (h3 / r1 + i + 1.5) / size;

return
(((texture(samplerUnit,
    vec3(p0.x, p0.y, p0.z)).r*r0.x-
    texture(samplerUnit,
        vec3(p1.x, p0.y, p0.z)).r*r1.x)*r0.y+
    texture(samplerUnit,
        vec3(p1.x, p1.y, p0.z)).r*r1.x-
    texture(samplerUnit,
        vec3(p0.x, p1.y, p0.z)).r*r0.x)*r1.y)*r0.z+
    ((texture(samplerUnit,
        vec3(p1.x, p0.y, p1.z)).r*r1.x -
    texture(samplerUnit,
        vec3(p0.x, p0.y, p1.z)).r*r0.x)*r0.y+
    texture(samplerUnit,
        vec3(p0.x, p1.y, p1.z)).r*r0.x-
    texture(samplerUnit,
        vec3(p1.x, p1.y, p1.z)).r*r1.x)*r1.y)*r1.z
) * minus_one_power(i);
}

```

This source code is easy to adapt to 1D and 2D interpolations, since the `vec3` variables need to be just replaced by `float` and `vec2` variables, respectively. Additionally, in 2D, we take just four bilinear texture fetches as we remove the z components of p_0 , p_1 , r_0 , and r_1 . Similarly, in 1D, we take just two linear texture fetches as we remove the y and z components of these vectors. Note that the 3D texture is uploaded after the modulation by the checkerboard pattern. In order to compensate this, the result is multiplied by `minus_one_power(i)`, which returns the parity of $i.x + i.y + i.z$. Following the suggestion of Ruijters [RMtHRS08], to maintain numerical precision, we calculate the sample positions on the fly instead of precalculating them in a lookup table as proposed by Sigg and Hadwiger [SH05]. However, a division by zero can still occur when vectors p_0 , and p_1 are calculated. This problem can be easily fixed if we remove the common roots from the polynomials that represent the numerators and the denominators. Therefore, instead of calculating weights h_0 , h_1 , h_2 , and h_3 explicitly, we calculate the necessary denominators d_0 and d_1 corresponding to p_0 and p_1 , respectively. Consequently, the simplified GLSL code is the following:

```

float CatmullRomSplineInterpolation(
    vec3 samplePosition) {
    vec3 s = samplePosition * size - 0.5;
    vec3 i = floor(s);
    vec3 t = s - i;
    vec3 t2 = t * t;

    vec3 d0 = 2.0 * t2 - 1.5 * t - 1.0;
    vec3 d1 = 2.0 * t2 - 2.5 * t - 0.5;

    vec3 p0 =
        ((1.5 * t2 - t - 1.0) / d0 + i - 0.5) / size;
    vec3 p1 =
        ((0.5 * t2 - 0.5 * t) / d1 + i + 1.5) / size;

    vec3 r0 = d0 * (t - 1.0);
    vec3 r1 = - d1 * t;
}

```

```

return
(((texture(samplerUnit,
    vec3(p0.x, p0.y, p0.z)).r*r0.x-
    texture(samplerUnit,
        vec3(p1.x, p0.y, p0.z)).r*r1.x)*r0.y+
    texture(samplerUnit,
        vec3(p1.x, p1.y, p0.z)).r*r1.x-
    texture(samplerUnit,
        vec3(p0.x, p1.y, p0.z)).r*r0.x)*r1.y)*r0.z+
    ((texture(samplerUnit,
        vec3(p1.x, p0.y, p1.z)).r*r1.x-
    texture(samplerUnit,
        vec3(p0.x, p0.y, p1.z)).r*r0.x)*r0.y+
    texture(samplerUnit,
        vec3(p0.x, p1.y, p1.z)).r*r0.x-
    texture(samplerUnit,
        vec3(p1.x, p1.y, p1.z)).r*r1.x)*r1.y)*r1.z
) * minus_one_power(i);
}

```

9. Mipmapping

Our method is easy to adapt to mipmapping. Instead of generating the entire mipmap automatically, the precalculated mipmap levels are modulated by a checkerboard pattern and uploaded to the texture memory separately. Catmull-Rom spline interpolation can be then performed on a specific mipmap level by using the following GLSL code:

```

vec4 CatmullRomSplineInterpolation(
    vec2 samplePosition, int lod)
{
    ivec2 size = textureSize(samplerUnit, lod);
    vec2 s = samplePosition * size - 0.5;
    vec2 i = floor(s);
    vec2 t = s - i;
    vec2 t2 = t * t;

    vec2 d0 = 2.0 * t2 - 1.5 * t - 1.0;
    vec2 d1 = 2.0 * t2 - 2.5 * t - 0.5;

    vec2 p0 =
        ((1.5 * t2 - t - 1.0) / d0 + i - 0.5) / size;
    vec2 p1 =
        ((0.5 * t2 - 0.5 * t) / d1 + i + 1.5) / size;

    vec2 r0 = d0 * (t - 1.0);
    vec2 r1 = - d1 * t;

    return (
        textureLod(samplerUnit,
            vec2(p0.x, p0.y), lod) * r0.x -
        textureLod(samplerUnit,
            vec2(p1.x, p0.y), lod) * r1.x) * r0.y +
        textureLod(samplerUnit,
            vec2(p1.x, p1.y), lod) * r1.x -
        textureLod(samplerUnit,
            vec2(p0.x, p1.y), lod) * r0.x) * r1.y
    ) * minus_one_power(i);
}

```

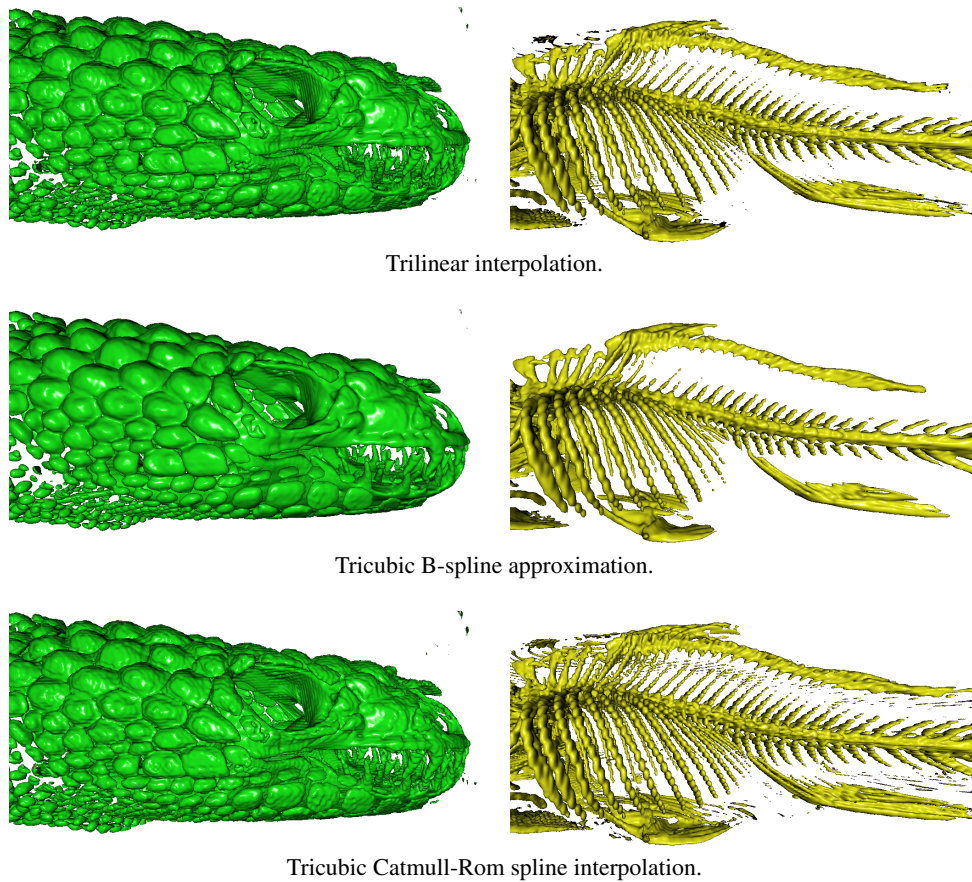


Figure 5: Reconstruction of CT data using trilinear interpolation, tricubic B-spline approximation, and Catmull-Rom spline interpolation. Note that the details are best preserved by the Catmull-Rom spline interpolation.

Previously, the resolution of the texture map was passed to the fragment shader as a uniform parameter `size`, but now it is evaluated for the actual mipmap level. A linear transition between two neighboring mipmap levels (see the result in Figure 6b), can be easily calculated in the fragment shader according to the current level of detail:

```
void main()
{
    vec2 lod = textureQueryLod(samplerUnit, texCoord);
    int i = int(floor(lod.x));
    float f = lod.x - i;
    vec3 color = mix(
        CatmullRomSplineInterpolation(texCoord, i).rgb,
        CatmullRomSplineInterpolation(texCoord, i+1).rgb,
        f);
    fragmentColor = vec4(color, 1);
}
```

10. Performance

Note that this paper focuses on an efficient GPU implementation of the Catmull-Rom spline interpolation. The superiority of the interpolating Catmull-Rom spline filtering over the approximating cubic

B-spline filtering has already been demonstrated in several comparative studies [MN88, ML94, MMY97, Csé08], including even a detailed frequency-domain analysis. Therefore, we show just a couple of practical examples, mainly for the sake of the validation of our implementation.

Figure 3 shows the upscaling of a 2D test signal using 2D texture mapping combined with bilinear interpolation, bicubic B-spline approximation, and bicubic Catmull-Rom spline interpolation. The bilinear interpolation introduces star-like artifacts and results in visually discontinuous rings. In contrast, the Catmull-Rom spline filtering provides a more continuous reproduction of the rings in a visual sense. Moreover, it leads to a much sharper image, unlike the bicubic B-spline filtering that introduces significant blurring.

Figure 4 shows the rendering of the classical Marschner-Lobb test signal [ML94] by using trilinear interpolation, tricubic B-spline approximation, and tricubic Catmull-Rom spline interpolation. We used the same ray casting framework, and to fairly compare the different reconstruction techniques, we adapted only the resampling function in the shader code to each filter. Note that the Catmull-Rom spline interpolation provides the best visual quality as it does not introduce so severe artifacts as the trilinear inter-

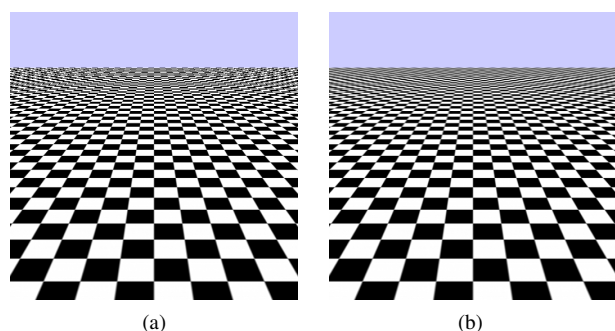


Figure 6: Texture sampling by using our fast Catmull-Rom spline interpolation with (b) and without (a) mipmapping.

polation, and does not have such a strong blurring effect as the tricubic B-spline approximation. The frame rates were measured on an nVidia GeForce GT 720M graphics card. We implemented the tricubic B-spline filtering by using the classical SH method, while for the Catmull-Rom spline filtering we implemented our method. Although both implementations require eight trilinear texture fetches, the Catmull-Rom spline filtering is slightly faster due to the simplified evaluation of the sample positions. Although the bottleneck is the number of texture fetches, the Catmull-Rom spline filtering is not eight times slower than the trilinear interpolation. Instead, it is just three times slower approximately. This is due to the fact that the eight trilinear samples are close to each other, so the cache coherence can be well exploited. We also rendered real-world CT data. Figure 5 shows the CT scans of an ancient snake and a carp. In both cases, the fine details are best reconstructed by the Catmull-Rom spline interpolation.

11. Conclusion

In this paper, we proposed a fast GPU implementation for Catmull-Rom spline interpolation. We pointed out that the classical SH algorithm [SH05], which has been previously applied for fast cubic B-spline filtering, cannot be trivially adapted to the Catmull-Rom spline as, due to its negative values, it does not meet a fundamental assumption of the algorithm. To appropriately handle the negative weights, we proposed a modulation of the original data by simply multiplying half of the data values by negative one. In 2D and 3D, this modulation results in a checkerboard pattern. As a major contribution, we redesigned the SH algorithm accordingly to be equivalent to the Catmull-Rom spline interpolation. Using our method, the Catmull-Rom spline interpolation can be evaluated as efficiently as the cubic B-spline approximation using the SH algorithm. This is a significant result as, more than ten years after the publication of the SH algorithm, an adaptation to the Catmull-Rom spline has not been published yet, although it has been reported to be needed in several practical applications.

Acknowledgements

This work has been supported by OTKA K-124124, VKSZ-14 PET/MRI 7T, and SCOPIA projects. The ancient snake data set is from the Digital Morphology (<http://www.digimorph.org>) data archive. Special thanks to Dr. Jessica A. Maisano for making this data set available to us.

References

- [BTU99] BLU T., THÉVENAZ P., UNSER M.: Generalized interpolation: Higher quality at no additional cost. In *Proceedings of IEEE International Conference on Image Processing* (1999), pp. 667–671. 2
- [CR74] CATMULL E., ROM R.: A class of local interpolating splines. *Computer Aided Geometric Design* (1974), 317–326. 1
- [Csé08] CSÉBFALVI B.: An evaluation of prefiltered reconstruction schemes for volume rendering. *IEEE Transactions on Visualization and Computer Graphics* 14, 2 (2008), 289–301. 1, 2, 7
- [HCSB] HADWIGER M., CAROLINE L., SCHARASCH H., BÜHLER K.: State of the art report 2004 on GPU-based segmentation. *Technical Report TR VRRIS 2004 17*, VRRIS Research Center, Vienna, Austria. 2
- [Key81] KEYS R. G.: Cubic convolution interpolation for digital image processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing ASSP-29*, 6 (1981), 1153–1160. 1
- [KW03] KRÜGER J., WESTERMANN R.: Acceleration techniques for GPU-based volume rendering. In *Proceedings of IEEE Visualization* (2003), pp. 38–45. 1, 2
- [LM05] LI S., MUELLER K.: Accelerated, high-quality refraction computations for volume graphics. In *Proceedings of Volume Graphics* (2005), pp. 73–81. 1
- [ML94] MARSCHNER S., LOBB R.: An evaluation of reconstruction filters for volume rendering. In *Proceedings of IEEE Visualization* (1994), pp. 100–107. 1, 7
- [MMMY97] MÖLLER T., MACHIRAJU R., MUELLER K., YAGEL R.: Evaluation and design of filters using a Taylor series expansion. *IEEE Transactions on Visualization and Computer Graphics* 3, 2 (1997), 184–199. 1, 2, 7
- [MN88] MITCHELL D., NETRAVALI A.: Reconstruction filters in computer graphics. In *Proceedings of SIGGRAPH* (1988), pp. 221–228. 1, 7
- [RMtHRS08] RUIJTERS D., M TER HAAR ROMENY B., SUETENS P.: Efficient GPU-based texture interpolation using uniform B-splines. *Journal of Computer Tools* 13, 4 (2008), 61–69. 6
- [RT12] RUIJTERS D., THÉVENAZ P.: GPU prefilter for accurate cubic B-spline interpolation. *The Computer Journal* 55, 1 (2012), 15–20. 1, 2
- [SF71] STRANG G., FIX G.: A Fourier analysis of the finite element variational method. In *Constructive Aspects of Functional Analysis* (1971), pp. 796–830. 1
- [SH05] SIGG C., HADWIGER M.: Fast third-order texture filtering. In *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation* (2005), Matt Pharr (ed.), Addison-Wesley, pp. 313–329. 1, 4, 6, 8
- [SK15] SZIRMAY-KALOS L.: Filtering and gradient estimation for distance fields by quadratic regression. *Periodica Polytechnica Electrical Engineering and Computer Science* 59, 4 (2015), 175–180. 1
- [SKSS08] SZIRMAY-KALOS L., SZÉCSI L., SBERT M.: *GPU-Based Techniques for Global Illumination Effects*. Morgan and Claypool Publishers, San Rafael, USA, 2008. 1
- [SSKH10] SHAMS R., SADEGHI P., KENEDY R. A., HARTLEY R. I.: A survey of medical image registration on multicore and the GPU. *IEEE Signal Processing Magazine* 27, 2 (2010), 50–60. 1
- [TBU00] THÉVENAZ P., BLU T., UNSER M.: Interpolation revisited. *IEEE Transactions on Medical Imaging* 19, 7 (2000), 739–758. 1
- [TUSK09] TÓTH B., UMENHOFFER T., SZIRMAY-KALOS L.: Efficient post-processing with importance sampling. In *ShaderX 7*, Engel W., (Ed.), Charles River Media, 2009. 1
- [UTY95] UNSER M., THÉVENAZ P., YAROSLAVSKY L.: Convolution-based interpolation for fast, high-quality rotation of images. *IEEE Transactions on Image Processing* 4, 10 (1995), 1371–1381. 1
- [WE98] WESTERMANN R., ERTL T.: Efficiently using graphics hardware in volume rendering applications. In *Proceedings of SIGGRAPH* (1998), pp. 169–176. 1, 2